Multilayer Perceptron

Dataset: California Housing Dataset

Data Set Characteristics:
Number of Instances: 20,640
Number of Attributes: 8 numeric, predictive attributes and 1 target variable

Attribute Information:
MedInc: Median income in the block group
HouseAge: Median house age in the block group
AveRooms: Average number of rooms per household
AveBedrms: Average number of bedrooms per household
Population: Population of the block group
AveOccup: Average number of household members
Latitude: Latitude of the block group
Longitude: Longitude of the block group

In [77]:
```
!pip install numpy scikit-learn matplotlib -i https://pypi.tuna.tsinghua.edu.cn/
```

```
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Requirement already satisfied: numpy in e:\anaconda\lib\site-packages (1.26.4)
Requirement already satisfied: scikit-learn in e:\anaconda\lib\site-packages (1.
5.1)
Requirement already satisfied: matplotlib in e:\anaconda\lib\site-packages (3.9.
2)
Requirement already satisfied: scipy>=1.6.0 in e:\anaconda\lib\site-packages (fro
m scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in e:\anaconda\lib\site-packages (fr
om scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in e:\anaconda\lib\site-packa
ges (from scikit-learn) (3.5.0)
Requirement already satisfied: contourpy>=1.0.1 in e:\anaconda\lib\site-packages
(from matplotlib) (1.2.0)
Requirement already satisfied: cycler>=0.10 in e:\anaconda\lib\site-packages (fro
m matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in e:\anaconda\lib\site-packages
(from matplotlib) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in e:\anaconda\lib\site-packages
(from matplotlib) (1.4.4)
Requirement already satisfied: packaging>=20.0 in e:\anaconda\lib\site-packages
(from matplotlib) (24.1)
Requirement already satisfied: pillow>=8 in e:\anaconda\lib\site-packages (from m
atplotlib) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in e:\anaconda\lib\site-packages
(from matplotlib) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in e:\anaconda\lib\site-packa
ges (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in e:\anaconda\lib\site-packages (from py
thon-dateutil>=2.7->matplotlib) (1.16.0)
```

In [78]:
```
import pandas as pd
from sklearn.datasets import fetch_california_housing
```

```python
from sklearn.model_selection import train_test_split
import torch
from sklearn.neural_network import MLPRegressor
import numpy as np
import matplotlib.pyplot as plt
```

In [79]:
```python
housing = fetch_california_housing()

#complet the code here, you need to create a pandas dataframe with the data and
#the columns names should be the feature names
#the target should be a pandas series
#you can use the pd.DataFrame and pd.Series functions
#you can use the housing.data and housing.target variables
#you can use the housing.feature_names variable

data = pd.DataFrame(housing.data, columns=housing.feature_names)
target = pd.Series(housing.target)
```

In [80]:
```python
#complete the code here, you need to split the data and target into training and
x_train, x_test, y_train, y_test = train_test_split(data, target, test_size=0.2,
```

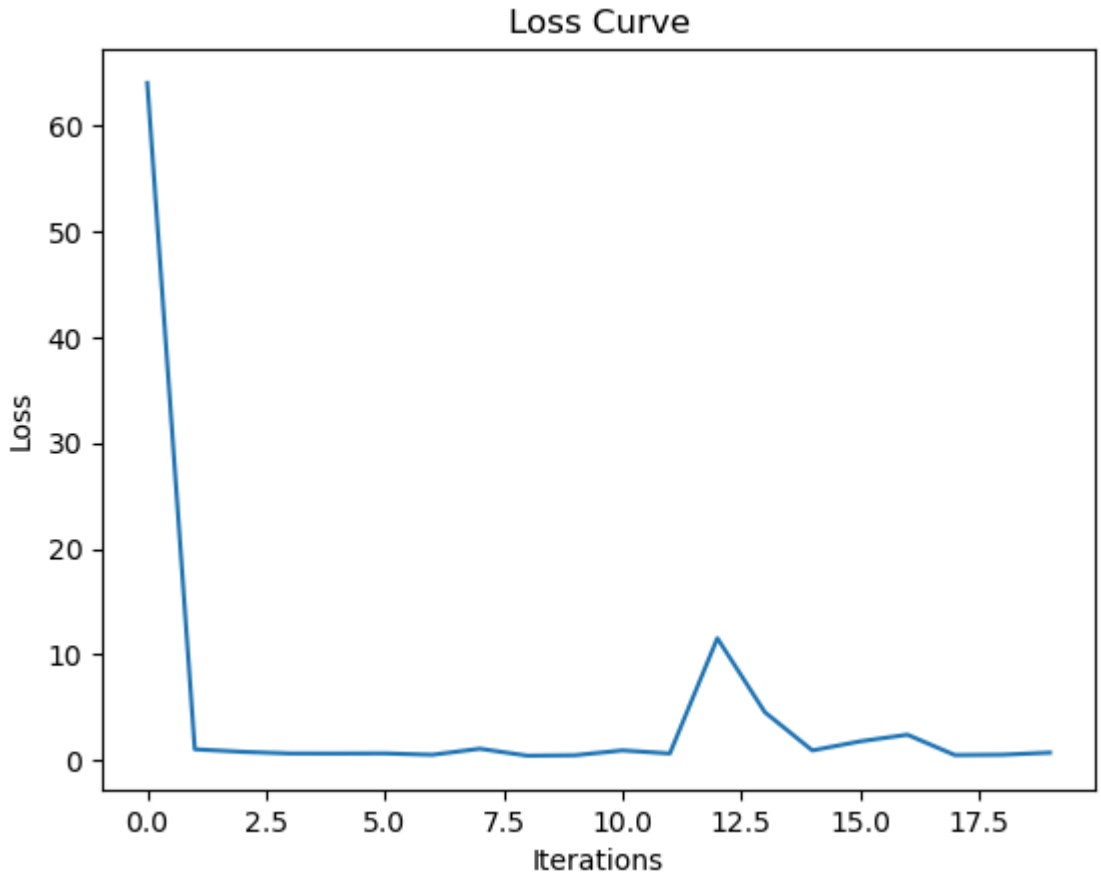
In [81]:
```python
# complete the code here, you need to create a MLPRegressor model
# you can use the MLPRegressor function
# you need to use the following parameters:

my_hidden_layer_sizes = (100, 50)
my_learning_rate_init = 0.001
my_activation = 'relu'
my_solver = 'adam'
my_max_iter = 1000

mlp = MLPRegressor(hidden_layer_sizes=my_hidden_layer_sizes, learning_rate_init
mlp.fit(x_train, y_train)
```

Out[81]:

MLPRegressor

```
MLPRegressor(hidden_layer_sizes=(100, 50), max_iter=1000)
```

In [82]:
```python
# complete the code here, you need to plot the loss curve
# you can use the mlp.loss_curve_ variable
# you can use the plt.plot function
plt.plot(mlp.loss_curve_)
plt.title('Loss Curve')
plt.xlabel('Iterations')
plt.ylabel('Loss')
plt.show()
```



In [83]:
```
#complete the code, you need to predict the test set and calculate the MAE, MSE
#you can use the mlp.predict function
#you can use the np.mean and np.abs functions
#you can use the np.sqrt function

y_pred = mlp.predict(x_test)

mae = np.mean(np.abs(y_test - y_pred))
mse = np.mean((y_test - y_pred)**2)
rmse = np.sqrt(mse)


print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
```

```
MAE: 0.6255959389796978
MSE: 0.7371517382858368
RMSE: 0.8585754121134828
```

In [84]:
```
# when you finish, please use 2pdf.com to transform this ipynb to pdf, and submi
```